Gespräch am MPI für Mikrostrukturphysik — Vorbereitung

Stand 23. September 2026. Für Malte, zum Lesen heute Abend und zum Mitnehmen. Aufbau: **1** Worum es geht in drei Sätzen · **2** Was ich mache, ausführlich erklärt · **3** Was ich beantrage · **4** Wie das MPI mich unterstützen kann · **5** Rechenbedarf in Zahlen · **6** Fragen, die ich stelle · **7** Fragen, die sie mir stellen werden, und gute Antworten · **8** Wo es zum MPI passt · **9** Fachbegriffe erklärt · **10** Was ich mitbringe.

---

1. Worum es geht — in drei Sätzen

Ich untersuche, ob Bauprinzipien aus der Quanteninformatik neuronale Netze **sparsamer** machen können — weniger Parameter, weniger Speicher, bei großer Breite weniger Rechenzeit. Dazu habe ich eine Quantenschaltung als trainierbare Schicht in ein neuronales Netz eingebaut, sie auf echter IBM-Quantenhardware laufen lassen und sie danach als rein klassisches Rechenverfahren nachgebaut, das auf normalen Prozessoren läuft. Jede Aussage prüfe ich gegen eine faire klassische Kontrolle, mit vorher festgelegten Kriterien — und für den letzten großen Schritt, ein Sprachmodell auf deutschem Text, fehlt mir die Rechenleistung.

---

2. Was ich mache — ausführlich

2.1 Die Ausgangsidee

Ein Quantencomputer mit **n Qubits** beschreibt seinen Zustand mit **2^n Zahlen** (Amplituden). Ein Gatter auf einem einzelnen Qubit wird aber nur durch **zwei oder drei Winkel** festgelegt — und wirkt trotzdem auf alle 2^n Zahlen gleichzeitig. Eine Quantenschaltung ist also eine Abbildung in einem riesigen Raum, die mit sehr wenigen Stellschrauben auskommt.

Zum Vergleich: Eine gewöhnliche („dichte“) Schicht eines neuronalen Netzes, die 256 Zahlen auf 256 Zahlen abbildet, hat 256 × 256 = **65.536** Gewichte. Eine Schaltung auf 8 Qubits ($2^8 = 256$) mit einer Lage Drehungen hat **16** Winkel.

Die Frage ist: **Reichen diese 16 Winkel, um dasselbe zu leisten?** Und wenn ja: warum?

2.2 Wie die Schicht funktioniert

1. **Kodieren.** Ein Bild (784 Pixel bei MNIST) wird von einer kleinen klassischen Schicht auf 256 Zahlen gebracht und auf Länge 1 normiert. Diese 256 Zahlen werden als die Amplituden eines 8-Qubit-Zustands benutzt (*Amplitudenkodierung*).
2. **Drehen.** Auf jedes Qubit wirken zwei Drehgatter (Ry und Rz) mit trainierbaren Winkeln. Optional folgt ein Ring aus CNOT-Gattern, der die Qubits *verschränkt*.
3. **Messen.** Für jedes der 8 Qubits wird der Erwartungswert $\langle Z \rangle$ gemessen. Das ist eine gewichtete Summe von **Betragsquadraten** der Amplituden (*Born-Regel*). Heraus kommen 8 Zahlen, aus denen eine letzte klassische Schicht die Ziffer (0–9) vorhersagt.

Trainiert wird alles zusammen mit Gradientenabstieg — die Gradienten fließen durch die Simulation der Schaltung hindurch. Das habe ich in einem selbst geschriebenen, differenzierbaren Simulator umgesetzt und gegen Qiskit geprüft.

2.3 Warum das klassisch schnell sein kann — die „Butterfly“-Struktur

Ein Gatter auf Qubit q mischt immer genau die Amplitudenpaare, deren Index sich nur im Bit q unterscheidet. Über alle Qubits hinweg ergibt das dasselbe Verbindungsmuster wie die **schnelle Fouriertransformation (FFT)** — ein „Schmetterlingsmuster“. Kosten: $d \cdot \log d$ statt d^2 Rechenschritte. Ich habe die Schaltung deshalb direkt als dieses klassische Verfahren programmiert (kein Simulator mehr), verifiziert auf $7 \cdot 10^{-7}$ gegen den Simulator.

Besonderheit gegenüber gewöhnlichen Butterfly-Netzen aus der Literatur: **Gewichtsteilung.** Weil ein Qubit-Gatter physikalisch auf *ein* Qubit wirkt, gilt derselbe Winkel für *alle* d/2 Paare einer Ebene. Bei 12 Qubits (d = 4.096): 120 Parameter statt 245.760 (ungeteiltes Butterfly) oder 16,8 Millionen (dicht). (Hinweis aus der

Literaturrecherche: Eine sehr ähnliche Schaltung mit ebenfalls ~80 Parametern wurde 2025 als „Quantum-PEFT“ veröffentlicht — die Bauform ist also bekannt; mein Beitrag liegt in der Prüfung und Erklärung, siehe Abschnitt 7.)

2.4 Was ich herausgefunden habe — ehrlich, mit Grenzen

Geschwindigkeit (auf meinem Prozessor, Ryzen 5 3600): Ab 16.384 Dimensionen ist die Butterfly-Schicht schneller als eine dichte (gemessen 1,59×, mit optimierter Umsetzung 2,31×, jeweils gepaart über 7 Messreihen). Darunter verliert sie deutlich. Der Übergang entsteht, weil die dichte Schicht dort aus dem Cache fällt (Sprung um Faktor 9,1). **Auf der GPU ist die Struktur dagegen viel langsamer** — sie besteht aus vielen kleinen Rechenschritten, und GPUs sind für wenige große gebaut.

Parameter-Sparsamkeit (MNIST, Fashion-MNIST): 16–80 Winkel erreichen in der Mittelschicht 0,934 Genauigkeit auf MNIST. Ein klassisches quadratisches Modell mit 2.048 Parametern liegt bei 0,921. Das ist die stärkste Zahl des Projekts.

Eine Einschränkung, die ich gerade selbst gefunden habe (23. September): Die klassische Vergleichsgruppe, gegen die ich bisher „+0,093 bei 848-fach weniger Parametern“ berichtet habe, ist zu schwach — sie ist sogar schlechter als ein rein lineares Modell. Eine Stichprobe mit einem gewöhnlichen ReLU-Netz gleicher Bauform liegt bei ~0,925. Ich rechne das jetzt sauber und vorregistriert nach (Q-020). Wahrscheinliche ehrliche Form der Aussage: **Gleiche Genauigkeit wie ein klassisches Netz mit tausendfach mehr Parametern in dieser Schicht** — nicht „besser“. Das sage ich im Gespräch lieber selbst, bevor es jemand anders merkt.

Woher der Effekt kommt: Rund 87–92 % stammen aus der *quadratischen Auslese* (Born-Regel als Nichtlinearität) — das ist klassisch nachbaubar. Die Verschränkung hat in meinen bisherigen Messungen nichts beigetragen (so auch in der Literatur: Bowles, Ahmed, Schuld 2024). Diese Messung muss ich aber ohne den vorgeschalteten Encoder wiederholen, weil der die Arbeit übernehmen kann (Lehre aus meinem Lauf Q-018).

Quantenkerne: Sieben quanteninspirierte Mechanismen, sieben faire Tests, keiner hat bestanden. Gemeinsame Ursache, gemessen: In 2^n Dimensionen sind zufällige Zustände fast orthogonal — die Ähnlichkeit halbiert sich je Qubit. Das deckt sich mit Thanasilp et al. (Nature Communications 2024) zur „exponentiellen Konzentration“.

Echte Hardware: Bell- und GHZ-Zustände sowie ein kleines trainiertes Modell liefen auf IBM-Prozessoren (`ibm_marrakesh`, `ibm_fez`). Fehlerrate wächst mit der Tiefe (1,57 % → 2,06 %, p = 0,043). Einschränkung: das Hardware-Modell war klein (20 Testbilder) und benutzte eine andere Kodierung als das Hauptmodell.

2.5 Wie ich arbeite — die Methodik

Das ist nach eigener Einschätzung der belastbarste Teil der Arbeit:

- **Vorregistrierung:** Hypothese, Test und alle möglichen Urteile stehen im Skript, bevor gerechnet wird.
- **Faire Kontrollen:** jeder Arm bekommt seine eigene, gesuchte Lernrate.
- **Genug Wiederholungen** (15 Seeds), **gepaarte Tests** (Wilcoxon), **Holm-Korrektur**.
- **Äquivalenztest (TOST):** „nicht signifikant“ heißt nicht „gleich“ — Gleichwertigkeit muss man eigens nachweisen.
- **Positivkontrolle:** Jeder Aufbau muss zeigen, dass er einen bekannten Unterschied überhaupt sehen kann.
- **Vortest auf künstlichen Daten:** Die Auswertung wird vor dem Lauf mit erzwungenen Ergebnissen geprüft. Das hat bisher neun Auswertungsfehler vor dem Lauf gefunden.
- **Prüfwerkzeug q_audit:** rechnet jede Zahl im Tagebuch aus den Rohdaten nach (106 Prüfungen).
- Zurückgezogene Aussagen bleiben mit Begründung stehen.

2.6 Was als Nächstes kommt — und warum ich Rechenleistung brauche

1. **T-002 — Sprachmodell auf deutscher Wikipedia (397 Mio. Token).** Ein kleiner Transformer (5,3 Mio. Parameter), bei dem Teile durch die Quantenstruktur ersetzt werden. Vier Varianten × 8 Wiederholungen = 32 Trainingsläufe. Auf meiner CPU unmöglich, auf einer GPU ~60–75 Stunden (vorsichtige Schätzung), auf 8 GPUs parallel rund 7–11 Stunden. Ergebnis: der Chatbot und die Antwort, ob die Struktur im Sprachmodell hält.
2. **Q-020 — die Kernaussage fair nachmessen** (CPU, mehrere Stunden).
3. **Q-011 — Orthogonalität in großer Tiefe** (CPU, 10–14 Stunden, halb fertig).

4. **Q-019 — Zeitreihen als zweite KI-Art** (CPU, ~5 Stunden).
5. **Ausblick:** die Quantenstruktur als sparsamer *Adapter* zum Nachtrainieren eines vorhandenen Sprachmodells (Vergleich mit LoRA und BOFT) — das braucht eine GPU.

2.7 Nur am Rande: ein getrenntes, klassisches Nebenprojekt (QIRE B)

Mein Hauptprojekt ist der **quantenphysikalische Ansatz (QIRE A)** — darum geht es bei der Rechenzeit. Nebenbei ist ein zweites, rein mathematisches Projekt entstanden, das ich nur kurz erwähne: Wie viele „Richtungen“ (Effektivrang) nutzt eine trainierte Schicht wirklich? In einem kontrollierten Versuch mit zwei Stellgrößen (Datendimension, Klassenzahl) zeigte sich: Der Rang wird von der Aufgabe gesetzt und ist sehr klein (1,4–4,7). Daraus folgt ein leicht umsetzbarer Kompressionsweg — eine fertig trainierte Schicht lässt sich auf Rang 64 kürzen, nachweislich ohne Verlust (32-fach weniger Parameter). Das braucht keine zusätzliche Rechenzeit.

---

3. Was ich beantrage — konkret

Priorität	Was	Umfang	Wofür
1	GPU-Rechenzeit (eine oder mehrere GPUs, z. B. A100/H100/RTX-Klasse)	~60–75 GPU-Stunden (Obergrenze der Schätzung), gern verteilt auf mehrere GPUs	T-002 Sprachmodell (8 Seeds × 4 Varianten + Lernratensuche)
2	CPU-Rechenknoten	~30–40 CPU-Stunden auf 6–8 Kernen	Q-020, Q-011, Q-019 — damit mein PC frei bleibt und Läufe nicht mit der Sitzung sterben
3	Eine fachliche Ansprechperson (Physik, gern Theorie oder Quanteninformation)	ein paar Gespräche bis Januar	Rückmeldung zu Deutung und Methodik: „Ist das quanten?“, „Ist der Test fair?“
4	optional: Zugang zu einem kleinen echten Datensatz aus dem Institut (Spektren, Zeitreihen, Messsignale)	wenige MB	eine echte Anwendung statt MNIST — Signale sind genau das, wofür die FFT-artige Struktur gebaut ist
5	optional: später GPU-Zeit für den Adapter-Versuch	~20–50 GPU-Stunden (Schätzung)	Nachtrainieren eines vortrainierten Sprachmodells mit der Quantenstruktur

Zeitrahmen: Die Anmeldung ist am **30. November**, die schriftliche Arbeit im **Januar 2027** (Mitte Januar). Ideal wären Ergebnisse bis **Mitte Dezember**.

Was ich **nicht** brauche: Geld, Laborplatz, Aufsicht über Nacht. Die Läufe sind vorbereitet, abbruchfest (setzen nach Absturz fort) und schreiben Rohdaten nach jedem Abschnitt.

---

4. Wie das MPI mich unterstützen kann — Möglichkeiten zum Ansprechen

- **Rechnerzugang:** Konto auf dem Institutscluster oder einem einzelnen GPU-Rechner; ein Mitarbeiter startet die Jobs, und ich bekomme die Ergebnisdateien — beides geht.
- **Betreuung:** bei Jugend forscht darf man Betreuer oder Unterstützer nennen. Fragen, ob jemand genannt werden möchte bzw. darf.
- **Rückmeldung zur schriftlichen Arbeit** im Dezember/Januar (Physik-Teil, Begriffe).
- **Daten und Fragestellung:** vielleicht gibt es im Institut Signal- oder Spektraldaten, an denen eine sparsame, FFT-artige Schicht sinnvoll wäre.
- **Hardware-Blick:** Das MPI forscht an energieeffizienter, neuromorpher und photonischer Hardware. Meine Schicht besteht nur aus 2×2-Drehungen mit geteilten Winkeln — genau die Bausteine, aus denen photonische Chips (Netze aus Mach-Zehnder-Interferometern) bestehen. Frage an sie: Ist eine Struktur mit „einer Einstellung pro Ebene statt tausender“ für solche Hardware interessant? (Idee, nicht geprüft.)
- **Praktikum / Schülerlabor / Folgeprojekt**, falls es das gibt.

---

5. Rechenbedarf in Zahlen (zum Zeigen)

Lauf	Rechner	Einzelteile	Zeit	Bemerkung
T-002 Hauptlauf	GPU	32 Trainingsläufe (4 Varianten × 8 Seeds), 13.000 Schritte, Batch 64 × 128 Token	Butterfly-Lauf ≈ 72 min (0,334 s/Schritt auf RTX 4090); dichte Läufe viel schneller	Läufe sind unabhängig → beliebig parallel
T-002 Lernratensuche	GPU	4 Varianten × ≥4 Lernraten, je 13.000 Schritte (volles Budget)	~16–30 GPU-h	nötig nach Protokoll
T-002 gesamt	GPU	48–60 Läufe	~60–75 GPU-h auf einer Karte (Schätzung, eher Obergrenze)	8 GPUs → ~7–11 h, 16 GPUs → ~4–7 h
Q-020	CPU	[wird gerade geschätzt]	mehrere Stunden	faire Neuprüfung der Kernaussage
Q-011 Rest	CPU	Tiefe 16 + Fashion-MNIST	10–14 h	Unitarität in der Tiefe
Q-019	CPU	5 Varianten × 2 Bedingungen × 15 Seeds	4,6–5,5 h	Zeitreihen

Technische Voraussetzungen: Python 3.11, PyTorch mit CUDA, numpy/scipy/pandas/tokenizers. Speicher: ~1 GB für den Korpus, das Modell selbst braucht weniger als 1 GB GPU-Speicher. Ohne Internet auf den Rechenknoten bringe ich den Korpus als Datei mit.

---

6. Fragen, die ich stelle

1. Welche Rechner gibt es — GPUs (Typ, Anzahl), CPU-Knoten? Läuft das über ein Warteschlangensystem (z. B. SLURM)?
2. Wie viele Stunden wären realistisch, und bis wann?
3. Bekomme ich selbst einen Zugang (ich bin unter 18 — braucht es eine Einverständniserklärung der Eltern, eine Nutzungsvereinbarung?) oder startet jemand die Jobs für mich?
4. Haben die Rechenknoten Internet, oder bringe ich die Daten mit?
5. Gibt es Regeln für Daten und Code (darf ich die Ergebnisse veröffentlichen / bei Jugend forscht zeigen)? Wie soll ich das MPI in der Arbeit erwähnen?
6. Gibt es jemanden, der sich den Physik-Teil anschauen würde?
7. Gibt es Daten im Institut, bei denen eine sparsame, FFT-artige Schicht nützlich wäre?

---

7. Fragen, die sie mir stellen werden — und gute Antworten

„Ist das ein echter Quantenvorteil?“ Nein, und das behaupte ich nicht. Ein beweisbarer Quantenvorteil beim Lernen existiert für *Quantendaten* (Huang et al., Science 2022), nicht für Bilder wie MNIST. Was ich zeige, ist ein *quanteninspirierter* Bauplan: Die Struktur kommt aus der Quantenmechanik, der Mechanismus ist klassisch erklärbar (quadratische Auslese). Das habe ich selbst gemessen.

„Bringt die Verschränkung etwas?“ In meinen bisherigen Messungen nicht. Allerdings saß dabei eine trainierbare Schicht davor, die die Arbeit übernehmen könnte. Deshalb messe ich es gerade ohne diese Schicht nach. Die Literatur (Bowles/Ahmed/Schuld 2024) findet dasselbe: Verschränkung zu entfernen schadet bei kleinen Aufgaben oft nicht.

„Warum nicht einfach LoRA oder ein kleineres Netz?“ Genau das ist die faire Kontrolle, gegen die ich messen muss. Im nächsten Schritt vergleiche ich die Struktur als Adapter direkt mit LoRA und BOFT bei gleicher Parameterzahl.

„Warum ist das auf der GPU langsamer?“ Die Butterfly-Struktur besteht aus vielen kleinen Schritten (log d Ebenen), eine dichte Schicht aus einer großen Matrixmultiplikation. GPUs sind für wenige große Operationen gebaut. Man bräuchte einen fusionierten Kernel (alle Ebenen in einem Durchlauf, Zustand im schnellen Speicher) — das ist offen. Auf der CPU gewinnt die Struktur ab 16.384 Dimensionen.

„Woher weißt du, dass das nicht Zufall ist?“ 15 Wiederholungen mit verschiedenen Zufallsstartwerten, gepaarte Tests, Holm-Korrektur, vorher festgelegte Kriterien, Positivkontrollen. Und ich habe mehrere eigene Befunde zurückgezogen, als sie bei mehr Wiederholungen nicht hielten.

„Wie groß ist dein Sprachmodell? Ist das ein Chatbot?“ 5,3 Millionen Parameter, trainiert auf deutscher Wikipedia. Es erzeugt grammatisch brauchbaren Text, ist aber kein Wissensspeicher. Zum echten Chatbot fehlt ein Nachtraining auf Dialogen.

„Was ist neu an deiner Arbeit?“ Ehrlich: Die Schaltung selbst ist nicht neu. Eine fast gleiche Schaltung wurde als „Quantum-PEFT“ veröffentlicht (Koike-Akino et al., ICLR 2025) und dort zum Nachtrainieren großer Sprachmodelle benutzt; das habe ich bei der Literaturrecherche gefunden. Neu an meiner Arbeit ist eher: (a) die systematischen, vorregistrierten Abgrenzungstests mit Positivkontrollen und Äquivalenztests für sieben quanteninspirierte Mechanismen — so etwas gibt es in der Literatur nur als Rangvergleich ohne Statistik (Bowles/Ahmed/Schuld 2024); (b) die gemessene Zerlegung, dass rund 87–92 % des Effekts aus der Born-Regel als Auslese kommen, und dass dieselbe Operation als Gate nichts bringt; (c) der eigene Weg, die Struktur auf GPUs schnell zu machen (Bündelung in wenige große Rechenschritte — gerade im Bau). Ausführlich: `A/LITERATUR_UND_ALLEINSTELLUNG.md`.

---

8. Wo es zum MPI passt

Das MPI Halle arbeitet an Materialien und Bauelementen für **energieeffizientes Rechnen** — Spintronik, neuromorphe Systeme, Nanophotonik, „Beyond von Neumann Computing“. Mein Thema ist dieselbe Frage von der Software-Seite: **Wie wenig Rechenaufwand braucht eine KI-Schicht?** Anknüpfungspunkt: Norm-erhaltende Schichten aus 2×2-Drehungen sind genau das, was photonische Chips physikalisch rechnen.

---

9. Fachbegriffe — einfach erklärt

Qubit — Quanten-Bit. Kann nicht nur 0 oder 1 sein, sondern eine Überlagerung beider. n Qubits zusammen werden durch 2^n Zahlen beschrieben.

Amplitude — eine dieser 2^n Zahlen. Ihr Betragsquadrat ist die Wahrscheinlichkeit, bei der Messung den zugehörigen Zustand zu sehen. Alle Betragsquadrate zusammen ergeben 1.

Superposition (Überlagerung) — der Zustand ist eine Mischung vieler Basiszustände gleichzeitig.

Verschränkung — die Qubits hängen so zusammen, dass man den Zustand nicht mehr als „Qubit 1 macht dies, Qubit 2 macht das“ beschreiben kann. Entsteht z. B. durch CNOT-Gatter.

Gatter — eine Operation auf Qubits, mathematisch eine Drehung im Zustandsraum. **Ry / Rz** sind Drehungen um die y- bzw. z-Achse, gesteuert durch einen Winkel. **CNOT** kippt ein Qubit, wenn ein anderes auf 1 steht.

Unitär / orthogonal — eine Abbildung, die Längen erhält (nichts wird verstärkt oder gedämpft). Quantengatter sind immer unitär; die reelle Version heißt orthogonal.

Born-Regel — die Regel, dass Messwahrscheinlichkeiten Betragsquadrate der Amplituden sind. In meiner Schicht ist das die Nichtlinearität (das „Quadrieren“).

Erwartungswert ⟨Z⟩ — der Mittelwert, den man misst, wenn man ein Qubit oft misst (+1 für „0“, −1 für „1“). Pro Qubit eine Zahl.

Amplitudenkodierung — Daten werden als Amplituden in einen Quantenzustand geschrieben: 256 Zahlen passen in 8 Qubits.

Parametrisierte Quantenschaltung / variationeller Ansatz — eine Schaltung mit einstellbaren Winkeln, die wie Gewichte eines neuronalen Netzes trainiert werden.

Quanteninspiriert — ein klassisches Verfahren, dessen Bauplan aus der Quantenphysik stammt, das aber auf normalen Computern läuft. Kein Quantencomputer nötig.

Quantenvorteil — eine Aufgabe, die ein Quantencomputer nachweisbar mit weniger Aufwand löst als jeder klassische Rechner. Für Bilddaten nicht nachgewiesen; für das Lernen aus Quantendaten schon (Huang et al. 2022).

Quantenkern — misst die Ähnlichkeit zweier Datenpunkte als Überlapp ihrer Quantenzustände. Scheitert bei vielen Qubits an der **Konzentration**: alle Zustände werden einander gleich unähnlich.

Barren Plateau — bei großen Schaltungen werden die Gradienten exponentiell klein; das Training kommt nicht vom Fleck.

NISQ — „noisy intermediate-scale quantum“: heutige Quantencomputer, verrauscht und klein.

Neuronales Netz, Schicht, Gewicht/Parameter — ein Netz besteht aus Schichten; jede Schicht hat Zahlen (Gewichte), die beim Training eingestellt werden. „Parameter“ = alle diese Zahlen.

Dichte Schicht — jede Eingabe ist mit jeder Ausgabe verbunden: d × d Gewichte.

Encoder / Kopf — die Schicht vor bzw. nach dem untersuchten Mittelteil.

Engstelle (Bottleneck) — eine Stelle mit sehr wenigen Kanälen (bei mir 8), durch die alle Information muss.

Butterfly-Struktur — Rechenmuster der schnellen Fouriertransformation: in $\log_2(d)$ Ebenen wird jeweils paarweise gemischt. Kosten $d \cdot \log d$ statt d^2.

FFT (schnelle Fouriertransformation) — zerlegt ein Signal in seine Frequenzen, in $d \cdot \log d$ Schritten.

Gewichtsteilung — mehrere Stellen im Netz benutzen denselben Parameter. Bei mir: ein Winkel für alle Paare einer Ebene.

Kronecker-Produkt — setzt kleine Matrizen zu einer großen zusammen. Eine Schaltung ohne Verschränkung ist genau ein Kronecker-Produkt von 2×2-Drehungen.

Effektivrang — wie viele „Richtungen“ eine Schicht tatsächlich nutzt, auch wenn sie theoretisch viel mehr hätte.

Transformer, FFN, Q/K/V, Aufmerksamkeit (Attention) — die Bauweise moderner Sprachmodelle. Das FFN ist der Rechenblock in jeder Lage; Q/K/V sind die drei Projektionen, mit denen das Modell entscheidet, welche früheren Wörter wichtig sind.

Token / BPE — Text wird in Stücke (Tokens) zerlegt; BPE ist das Verfahren dafür. Mein Wortschatz hat 4.096 Stücke.

Perplexität / nats — Maß dafür, wie überrascht ein Sprachmodell vom nächsten Token ist. Kleiner ist besser. nats = Kreuzentropie mit natürlichem Logarithmus.

Adapter, LoRA, BOFT — Verfahren, um ein fertig trainiertes großes Modell mit wenigen zusätzlichen Parametern an eine neue Aufgabe anzupassen. LoRA benutzt kleine Matrizen niedrigen Rangs, BOFT orthogonale Butterfly-Matrizen.

GPU / CPU / Cache / Kernel — GPU: Grafikprozessor, sehr schnell für große, gleichförmige Rechnungen. CPU: normaler Prozessor. Cache: kleiner, sehr schneller Zwischenspeicher im Prozessor. Kernel: ein einzelnes Rechenprogramm auf der GPU; viele kleine Kernel sind langsam.

SLURM / Job-Array — Warteschlangensystem auf Rechenclustern; ein Job-Array startet viele gleichartige Läufe (z. B. je Seed einen) auf einmal.

Seed — Startwert des Zufallsgenerators. Verschiedene Seeds = unabhängige Wiederholungen.

Lernrate — wie groß die Schritte beim Training sind. Zu groß oder zu klein verfälscht jeden Vergleich.

p-Wert — wie wahrscheinlich ein mindestens so großer Unterschied wäre, wenn in Wahrheit keiner existiert. Unter 0,05 heißt „signifikant“.

Wilcoxon-Test — prüft, ob ein Verfahren in gepaarten Wiederholungen systematisch besser ist, ohne Normalverteilung vorauszusetzen.

Holm-Korrektur — macht p-Werte strenger, wenn man mehrere Vergleiche gleichzeitig testet.

TOST (Äquivalenztest) — weist nach, dass zwei Verfahren *innerhalb einer vorher festgelegten Grenze gleich* sind. „Nicht signifikant“ allein reicht dafür nicht.

Vorregistrierung — Hypothese, Test und Urteil werden festgelegt, *bevor* Daten entstehen. Schützt vor nachträglichem Zurechtbiegen.

Positivkontrolle / Auflösungsprüfung — ein Vergleich mit bekanntem Ausgang. Findet der Aufbau nicht einmal den, kann er die eigentliche Frage nicht beantworten.

Pseudoreplikation — Messungen als unabhängig zählen, obwohl sie es nicht sind (z. B. zehn Seeds derselben Versuchsbedingung als zehn Bedingungen). Macht p-Werte viel zu klein.

---

10. Was ich mitbringe

- Laptop mit der Standdemo (`A/demo`, offline: Ziffer zeichnen, drei Modelle vergleichen)
- Diese Seite ausgedruckt, dazu die Rechenbedarf-Tabelle (Abschnitt 5)
- Die Einordnung in die Literatur (`A/LITERATUR_UND_ALLEINSTELLUNG.md`)
- Die Anleitung für den Cluster (`A/server_setup/cluster/`) — zeigt, dass alles vorbereitet ist
- Auf dem Laptop: Korpus (`train.npy`, 795 MB) und Code, falls man gleich etwas starten will